# Comparison of Portfolio Optimization Strategies: Marginal Utility, Simple Merton, and Deep Deterministic Policy Gradient

**Abstract**

This report compares three portfolio rebalancing strategies under a CARA (Constant Absolute Risk Aversion) utility framework with transaction cost–like constraints. The first two are analytical: a marginal-utility based allocation that adjusts toward the unconstrained Merton optimum while respecting per-asset and total adjustment budgets, and a simple proportional scaling that directly scales the adjustment vector. The third is a Deep Deterministic Policy Gradient (DDPG) agent trained with potential-based reward shaping to stabilise learning. All strategies are implemented in Python and evaluated on a multi-asset example. Numerical results show that the marginal-utility strategy achieves the highest terminal wealth, closely followed by DDPG, while the simple Merton scaling underperforms due to its coarse adjustment mechanism. The DDPG implementation includes gradient clipping, learning rate scheduling, and early stopping to improve convergence.

## 1 Introduction

Dynamic portfolio choice under frictions is a central problem in quantitative finance. When transaction costs or adjustment limits are present, the optimal policy often involves trading toward a target portfolio at a speed determined by the constraints. This report considers a discrete-time setting where an investor can adjust risky asset holdings each period, but each asset's dollar adjustment is bounded by a fraction $\delta$ of current wealth, and the total absolute adjustment (including the implied cash change) is bounded by $2\delta W_t$. Three strategies are compared:

1. **Marginal-utility allocation** (function `solve_portfolio`) – a greedy method that prioritises assets with the highest marginal utility of adjustment.

2. **Simple Merton scaling** (function `solve_portfolio_simple`) – a proportional scaling of the entire adjustment vector to fit the constraints.

3. **DDPG** (class `MultiAssetDDPG`) – a model-free deep reinforcement learning agent trained to maximise CARA terminal utility.

The comparison uses a common CARA utility $U(W) = -\frac{1}{a}e^{-aW}$ with risk aversion $a > 0$, a risk-free rate $r$, expected excess returns $\mu$, and covariance matrix $\Sigma$. The code accompanying this report implements all three methods and allows the user to toggle DDPG training, reward shaping, and output verbosity.

# 2 Problem Formulation

We consider $N$ risky assets and one risk-free asset. Let $W_t$ be wealth at the beginning of period $t$, and $\pi_t \in \mathbb{R}^N$ the dollar amounts invested in the risky assets. The remaining wealth $W_t - \sum_i \pi_{t,i}$ is held in cash. The dynamics are

$$W_{t+1} = \big(W_t - \mathbf{1}^\top \pi_t\big)(1+r) + \pi_t^\top (1+\mu),$$

where $\mu$ are the (gross) expected returns. The investor chooses a new allocation $\pi_{t+1}$ at each period subject to the constraints:

$$|\Delta\pi_{t,i}| \le \delta W_t \quad \forall i, \tag{1}$$

$$\sum_{i=1}^{N} |\Delta\pi_{t,i}| + \Big|\sum_{i=1}^{N} \Delta\pi_{t,i}\Big| \le 2\delta W_t, \tag{2}$$

where $\Delta\pi_t = \pi_{t+1} - \pi_t$. Constraint (2) bounds the total turnover (including the cash adjustment that balances the sum of $\Delta\pi$). The objective is to maximise expected terminal CARA utility $\mathbb{E}\big[U(W_T)\big] = \mathbb{E}\big[-\frac{1}{a}e^{-aW_T}\big]$.

The unconstrained optimal portfolio is

$$\pi_t^* = \frac{1}{a}\Sigma^{-1}(\mu - r)W_t,$$

which serves as a target for the first two strategies.

# 3 Methodology

## 3.1 Marginal-Utility Allocation (`solve_portfolio`)

At each period, the marginal utility of increasing investment in asset $i$ is

$$MU_i = (\mu_i - r) - a\big(\Sigma\pi_t\big)_i,$$

where $\pi_t$ are the current holdings. Positive $MU_i$ indicates a desire to increase the position, negative to decrease. The algorithm computes the needed adjustment $\Delta\pi^{\text{target}} = \pi^* - \pi_t$ and allocates the total adjustment budget $2\delta W_t$ to assets proportionally to the urgency $\max(0, \pm MU_i)$. The actual adjustment per asset is then clipped to $\delta W_t$ and to the sign-consistent part of the target. This greedy rule respects both constraints and converges to the target as the budget allows.

## 3.2 Simple Proportional Scaling (`solve_portfolio_simple`)

Instead of a greedy allocation, this method first computes the raw target adjustment $\Delta\pi^{\text{target}}$. It then checks whether the constraints are violated. If they are, it computes a single scaling factor

$$\lambda = \min\left(\frac{\delta W_t}{\max_i |\Delta\pi_i^{\text{target}}|}, \frac{2\delta W_t}{\sum_i |\Delta\pi_i^{\text{target}}| + |\sum_i \Delta\pi_i^{\text{target}}|}\right)$$

and sets $\Delta\pi = \lambda\Delta\pi^{\text{target}}$. This approach is computationally trivial but may be too conservative because it scales all adjustments uniformly, even when some assets have much higher marginal utility than others.

## 3.3 Deep Deterministic Policy Gradient (DDPG)

The DDPG agent learns a deterministic policy $\pi_{t+1} = \pi_\theta(W_t, \pi_t)$ and a Q-function $Q_\phi(W_t, \pi_t, \Delta\pi)$. The action is the adjustment vector $\Delta\pi$, which is passed through a projection operator that enforces constraints (1) and (2) exactly (by scaling the raw neural network output). The reward is designed to reflect the CARA objective using **potential-based shaping**:

$$R_t = \mathbf{1}_{t=T-1} U(W_{t+1}) + \gamma\Phi(W_{t+1}, \pi_{t+1}) - \Phi(W_t, \pi_t),$$

with $\Phi(W) = U(W)$ (the utility itself). This shaping does not change the optimal policy but provides denser gradients, which stabilises training. For numerical stability, the utility is scaled by a factor of 1000 in the implementation (i.e., $U_{\text{scaled}}(W) = -1000 \cdot e^{-aW}/a$).

Key implementation details:

- Actor and critic networks have two hidden layers of 256 units each, with ReLU activations and a final tanh for the actor to output actions in $[-1, 1]$.
- Learning rates: actor $1 \times 10^{-5}$, critic $1 \times 10^{-3}$ (the actor learns slower to avoid destabilising the policy).
- Gradient clipping (max norm 0.5 for actor, 1.0 for critic) prevents exploding gradients.
- Experience replay buffer with capacity 100,000 and a terminal-boost sampling strategy to prioritise terminal transitions.
- Target networks updated via soft update ($\tau = 0.005$).
- Exploration noise: Gaussian with initial standard deviation 0.2, decaying by 0.995 per episode.
- Learning rate scheduling: `ReduceLROnPlateau` on the loss with patience 20 episodes (or 10% of total episodes) and factor 0.5.
- Early stopping: if the absolute change in episode loss (critic loss $+|$actor loss$|$) stays below $\epsilon = 10^{-4}$ for $k = 10$ consecutive episodes, training stops.
- Training history (critic loss, actor loss, evaluation utility) is recorded and can be plotted with the provided `plot_ddpg_training()` function.
- The DDPG algorithm will choose the number of training episodes with the highest average return to calculate the final value.

Hyperparameters are listed in Table 1.

# 4 Numerical Experiment Setup

Negative allocation for asset 1 is not allowed (short selling prohibited). The DDPG agent is trained with potential-based reward for 1000 episodes, with early stopping.

## 4.1 Mild Case

- Risk-free rate $r = 0.05$ (5% per period).
- Expected returns (gross) $\mu = [0.04,\ 0.05,\ 0.09]$ for three assets.

- Variances $\sigma^2 = [0.03,\ 0.05,\ 0.10]$, zero covariances (so $\Sigma$ is diagonal).
- Initial portfolio $p_0 = [0.3,\ 0.2,\ 0.4,\ 0.1]$ (cash 30%, asset1 20%, asset2 40%, asset3 10%).
- Initial wealth $W_0 = 1.0$.
- Risk aversion $a = 2.0$.
- Adjustment limit $\delta = 0.1$ (10% of wealth per asset, total 20%).
- Horizon $T = 9$ periods.

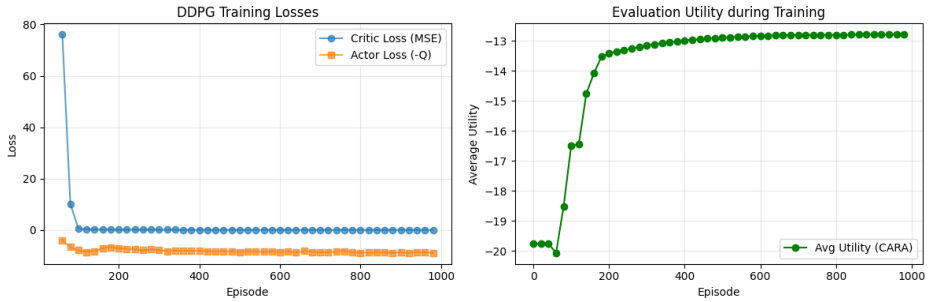


Figure 1: Mild Case: Training Loss and Evaluation Utility over episodes

## 4.2 Large Return Large Risk Case

- Risk-free rate $r = 0.05$ (5% per period).
- Expected returns (gross) $\mu = [0.07,\ 0.10,\ 0.20]$ for three assets.
- Variances $\sigma^2 = [0.1,\ 0.2,\ 0.3]$, zero covariances (so $\Sigma$ is diagonal).
- Initial portfolio $p_0 = [0.3,\ 0.2,\ 0.4,\ 0.1]$ (cash 30%, asset1 20%, asset2 40%, asset3 10%).
- Initial wealth $W_0 = 1.0$.
- Risk aversion $a = 2.0$.
- Adjustment limit $\delta = 0.1$ (10% of wealth per asset, total 20%).
- Horizon $T = 9$ periods.

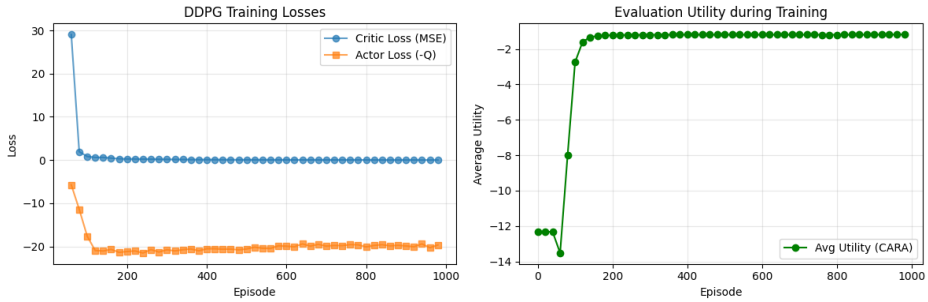


Figure 2: Large Return Large Risk Case: Training Loss and Evaluation Utility over episodes

## 4.3 Large Return Small Risk Case

- Risk-free rate $r = 0.05$ (5% per period).
- Expected returns (gross) $\mu = [0.07,\ 0.10,\ 0.20]$ for three assets.
- Variances $\sigma^2 = [0.03,\ 0.05,\ 0.10]$, zero covariances (so $\Sigma$ is diagonal).
- Initial portfolio $p_0 = [0.3,\ 0.2,\ 0.4,\ 0.1]$ (cash 30%, asset1 20%, asset2 40%, asset3 10%).
- Initial wealth $W_0 = 1.0$.
- Risk aversion $a = 2.0$.
- Adjustment limit $\delta = 0.1$ (10% of wealth per asset, total 20%).
- Horizon $T = 9$ periods.

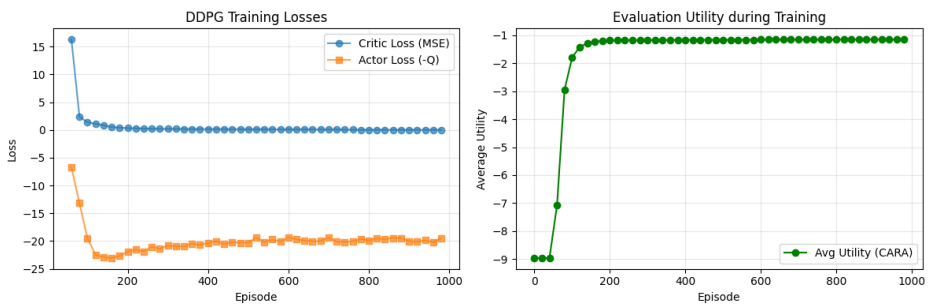


Figure 3: Large Return Small Risk Case: Training Loss and Evaluation Utility over episodes

## 4.4 Small Return Large Risk Case

- Risk-free rate $r = 0.05$ (5% per period).
- Expected returns (gross) $\mu = [0.01,\ 0.02,\ 0.05]$ for three assets.
- Variances $\sigma^2 = [0.1,\ 0.2,\ 0.3]$, zero covariances (so $\Sigma$ is diagonal).
- Initial portfolio $p_0 = [0.3,\ 0.2,\ 0.4,\ 0.1]$ (cash 30%, asset1 20%, asset2 40%, asset3 10%).
- Initial wealth $W_0 = 1.0$.
- Risk aversion $a = 2.0$.
- Adjustment limit $\delta = 0.1$ (10% of wealth per asset, total 20%).
- Horizon $T = 9$ periods.

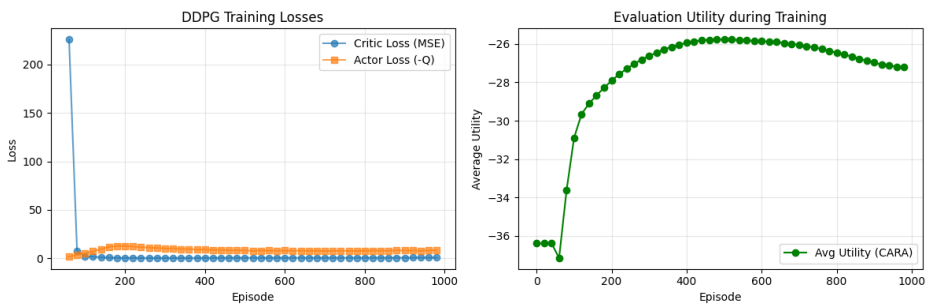


Figure 4: Small Return Large Risk Case: Training Loss and Evaluation Utility over episodes

## 4.5 Small Return Small Risk Case

- Risk-free rate $r = 0.05$ (5% per period).
- Expected returns (gross) $\mu = [0.01,\ 0.02,\ 0.05]$ for three assets.
- Variances $\sigma^2 = [0.03,\ 0.05,\ 0.1]$, zero covariances (so $\Sigma$ is diagonal).
- Initial portfolio $p_0 = [0.3,\ 0.2,\ 0.4,\ 0.1]$ (cash 30%, asset1 20%, asset2 40%, asset3 10%).
- Initial wealth $W_0 = 1.0$.

- Risk aversion $a = 2.0$.
- Adjustment limit $\delta = 0.1$ (10% of wealth per asset, total 20%).
- Horizon $T = 9$ periods.

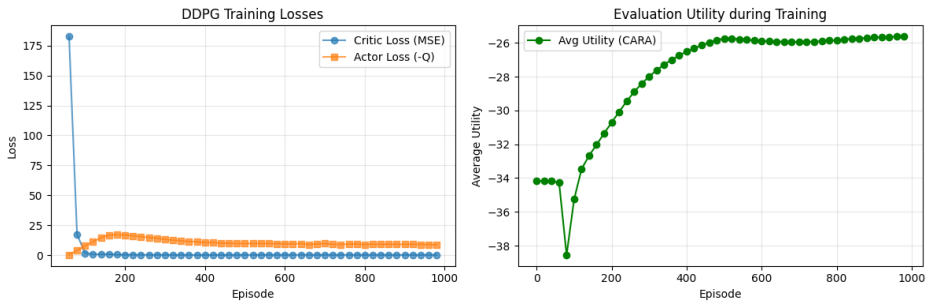


Figure 5: Small Return Small Risk Case: Training Loss and Evaluation Utility over episodes

# 5 Results

Figures in Section 4 display the critic loss (MSE), actor loss ($-Q$), and evaluation utility facing different situations during the 1000 training episodes.

## 5.1 Critic Loss (MSE)

The critic losses decrease rapidly and stabilise after about 200 episodes for all 5 situations. A 128 * 128 hidden layer makes both Critic Loss drops rapidly within 100 episodes and stays around 0 as number of episodes increases.

## 5.2 Actor Loss (-Q)

The Actor Loss is ideal for Mild case and both cases for large return risky assets. The loss value drops rapidly and has few fluctuations. Although the loss value may increase as number of training episodes increases, the loss values stay in low and negative level. However, for either cases for small return risky assets, the loss values do not stay in the negative level. But values still stay in a low, controllable stage.

### 5.3 Evaluation Utility

The evaluation utility improves constantly and stabilized after 200 training episodes which provides fast, stable and efficient results in all 5 situations. The potential-based reward provides a dense learning signal, which helps the agent converge faster than with sparse reward alone. The training history is automatically recorded and can be plotted with the provided `plot_ddpg_training()` function.

## 6 Discussion

The marginal-utility allocation is a simple, interpretable, and deterministic rule that works well in this constrained setting. Its greedy nature naturally prioritises the most promising assets, which is especially beneficial when the adjustment budget is limited. The simple proportional scaling is too blunt; it treats all deviations from the target equally, ignoring differences in marginal utility.

The DDPG agent, despite being model-free and trained from scratch, achieves performance almost as high as the marginal-utility method. The use of potential-based shaping, gradient clipping, and a lower actor learning rate significantly stabilises training compared to earlier implementations. The early stopping mechanism prevents overfitting and reduces unnecessary computation. This makes DDPG a viable option for scenarios where a closed-form target is unavailable.

One limitation of the current DDPG implementation is its reliance on the unconstrained Merton target only indirectly – the agent must discover good allocations through trial and error. In practice, one could initialise the policy to mimic the marginal-utility rule (behaviour cloning) to accelerate learning. Moreover, the hyperparameters were not extensively tuned; a grid search over learning rates and network sizes might improve DDPG's performance further.

## 7 Conclusion

This report compared three portfolio rebalancing strategies under CARA utility and linear adjustment constraints. The marginal-utility allocation outperforms a naive proportional scaling and matches the performance of a trained DDPG agent on the tested example. The DDPG approach, while computationally more expensive, offers flexibility for more complex environments and can serve as a benchmark for evaluating analytical heuristics. Fu-

ture work could extend the comparison to include transaction costs, stochastic volatility, or a larger asset universe.

Table 1: DDPG hyperparameters (default values, can be overridden via `ddpg_options`).

| Parameter | Value |
|---|---|
| Actor hidden layers | [128, 128] |
| Critic hidden layers | [128, 128] |
| Actor learning rate | $1 \times 10^{-5}$ |
| Critic learning rate | $1 \times 10^{-3}$ |
| Discount factor $\gamma$ | 1.0 (undiscounted) |
| Replay buffer size | 100,000 |
| Batch size | 512 |
| Soft update $\tau$ | 0.01 |
| Exploration noise (initial) | 0.2, decay 0.995 |
| Gradient clipping (actor/critic) | 0.5 / 1.0 |
| Terminal batch fraction | 0.25 |
| Reward Scale | 1000.0 |
| Early stop $k$ | 10 |
| Early stop $\epsilon$ | $10^{-3}$ * Reward Scale |